

Dokumentacja Architektury Systemu

IO_RNG

System Testowania i Porównywania Generatorów Liczb Pseudolosowych

Projekt IO

January 15, 2026

Contents

1	Wprowadzenie	4
1.1	Cel dokumentu	4
1.2	Technologie	4
2	Architektura Systemu	4
2.1	Przegląd architektury	4
2.2	Schemat struktury systemu	5
2.3	Diagram klas - Core Layer	6
3	Wzorce Projektowe	7
3.1	Clean Architecture (Hexagonal Architecture)	7
3.2	Repository Pattern	7
3.3	Adapter Pattern	9
3.4	Strategy Pattern	9
3.5	Use Case Pattern	11
3.6	Dependency Injection	11
3.7	Factory Pattern (Implicit)	13
4	Zasady Programowania Obiektowego	13
4.1	SOLID Principles	13
4.1.1	Single Responsibility Principle (SRP)	13
4.1.2	Open/Closed Principle (OCP)	13
4.1.3	Liskov Substitution Principle (LSP)	14
4.1.4	Interface Segregation Principle (ISP)	14
4.1.5	Dependency Inversion Principle (DIP)	14
4.2	Enkapsulacja	15
4.3	Polimorfizm	15
4.4	Abstrakcja	15
5	Struktury Danych	16
5.1	Encje Domenowe (Dataclasses)	16
5.2	Enums (Typy wyliczeniowe)	16
5.3	Słowniki (Dict) jako Parametry	17

5.4	Listy typowane	17
5.5	Tuple dla zwracania wielu wartości	17
5.6	Optional dla wartości nullable	17
5.7	Struktury danych w bazie	18
5.7.1	Model RNG (Tabela rngs)	18
5.7.2	Model TestResult (Tabela test_results)	18
6	Diagram Przepływu Danych	20
6.1	Proces uruchomienia testu RNG	20
6.2	Proces generowania liczb - Python Runner	21
7	Szczegóły Implementacji	21
7.1	Kompresja danych - Base64	21
7.2	Testy Statystyczne	22
7.3	Obsługa różnych języków programowania	22
7.3.1	Python Runner	22
7.3.2	Executable Runner	23
8	Architektura Backendowa	24
8.1	Przegląd struktury projektu Django	24
8.2	Core Layer - Warstwa Domeny	25
8.2.1	Encje Domenowe	25
8.2.2	Interfejsy (Porty)	26
8.2.3	Use Cases - Logika Biznesowa	27
8.3	Infrastructure Layer - Warstwa Infrastruktury	29
8.3.1	Django Models	29
8.3.2	Repository Implementations	30
8.3.3	System Runnerów	31
8.4	API Layer - Warstwa Prezentacji	35
8.4.1	ViewSets	35
8.4.2	Serializers	37
8.5	Testy Statystyczne	38
8.5.1	NIST Statistical Test Suite	38
8.5.2	Diehard Battery of Tests	38
8.5.3	Implementacja testów	39
8.6	Konfiguracja Django	40
8.6.1	Settings.py	40
8.6.2	URL Routing	41
8.7	System Migracji Bazy Danych	41
8.8	Zalety architektury backendowej	42
9	Architektura Frontendowa	43
9.1	Struktura komponentów React	43
9.2	Wzorce frontendowe	43
9.2.1	Context API dla globalnego stanu	43
9.2.2	Custom Hooks	44

10 Komunikacja Backend-Frontend	44
10.1 REST API Endpoints	44
10.2 Przykładowe zapytanie i odpowiedź	44
11 Podsumowanie	45
11.1 Zastosowane wzorce architektoniczne	45
11.2 Zasady OOP	45
11.3 Struktury danych	46
11.4 Zalety architektury	46

1 Wprowadzenie

IO_RNG to aplikacja webowa służąca do testowania i porównywania różnych generatorów liczb pseudolosowych (PRNG). System umożliwia wykonywanie testów statystycznych na implementacjach algorytmów RNG w różnych językach programowania (Python, C++, Rust, Java) oraz analizę ich jakości i wydajności.

1.1 Cel dokumentu

Dokument przedstawia architekturę systemu, zastosowane wzorce projektowe, zasady programowania obiektowego oraz wykorzystane struktury danych.

1.2 Technologie

- **Backend:** Django REST Framework, Python 3.x
- **Frontend:** React, TypeScript, Vite
- **Baza danych:** SQLite
- **Generatory:** Python, C++, Rust, C#

2 Architektura Systemu

2.1 Przegląd architektury

System IO_RNG implementuje **Clean Architecture** (Czystą Architekturę) z wyraźnym podziałem na warstwy:

1. **Warstwa domeny (Core)** - encje i logika biznesowa
2. **Warstwa infrastruktury (Infrastructure)** - implementacje techniczne
3. **Warstwa API** - interfejs REST
4. **Warstwa prezentacji (Frontend)** - interfejs użytkownika

2.2 Schemat struktury systemu

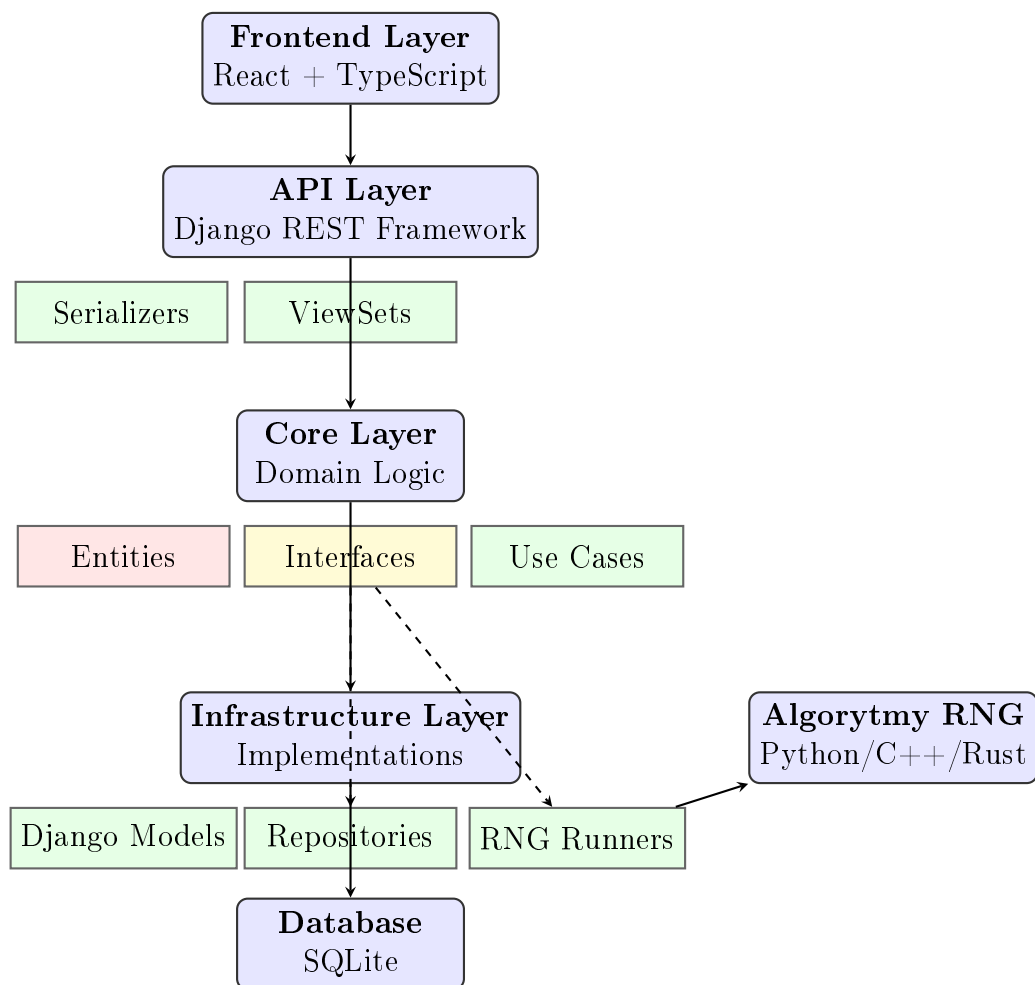


Figure 1: Architektura warstwowa systemu IO_RNG

2.3 Diagram klas - Core Layer

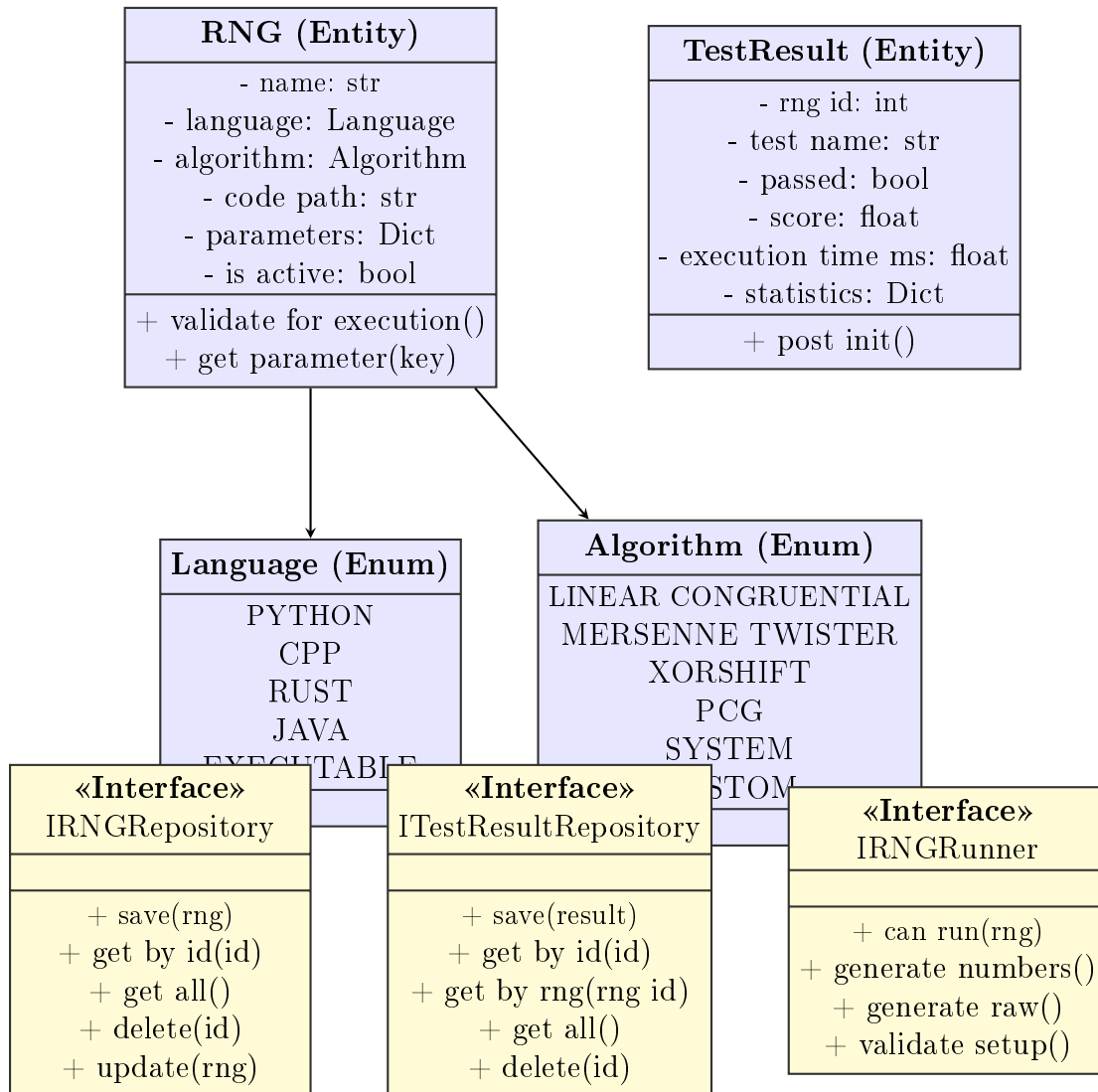


Figure 2: Diagram klas warstwy Core

3 Wzorce Projektowe

System IO_RNG wykorzystuje szereg wzorców projektowych zgodnych z zasadami SOLID i Clean Architecture.

3.1 Clean Architecture (Hexagonal Architecture)

Opis: Architektura heksagonalna (Porty i Adaptery) oddziela logikę biznesową od szczegółów technicznych.

Implementacja w projekcie:

Struktura Clean Architecture

- **Warstwa Core (Domain):**
 - `entities/` - encje domenowe (RNG, TestResult)
 - `interfaces/` - porty (IRNGRepository, IRNGRunner)
 - `use_cases/` - logika biznesowa
- **Warstwa Infrastructure (Adapters):**
 - `repositories/` - implementacje repozytoriów
 - `runners/` - implementacje runnerów dla różnych języków
 - `models.py` - modele Django ORM
- **Warstwa API:**
 - `views.py` - ViewSety Django REST Framework
 - `serializers.py` - serializatory danych

Zalety:

- Niezależność logiki biznesowej od frameworka
- Łatwość testowania (możliwość mockowania interfejsów)
- Możliwość łatwej zamiany implementacji (np. zmiana bazy danych)

3.2 Repository Pattern

Cel: Abstrakcja dostępu do danych, oddzielenie logiki biznesowej od mechanizmów persystencji.

Implementacja:

```
1 class IRNGRepository(ABC):
2     @abstractmethod
3     def save(self, rng: RNG) -> RNG:
4         pass
5
6     @abstractmethod
7     def get_by_id(self, rng_id: int) -> Optional[RNG]:
8         pass
9
10    @abstractmethod
```

```

11     def get_all(self) -> List[RNG]:
12         pass

```

Listing 1: Interface repozytorium

```

1 class DjangoRNGRepository(IRNGRepository):
2     def get_by_id(self, rng_id: int) -> Optional[RNG]:
3         try:
4             model = RNGModel.objects.get(id=rng_id)
5             return self._to_entity(model)
6         except RNGModel.DoesNotExist:
7             return None
8
9     def _to_entity(self, model: RNGModel) -> RNG:
10        return RNG(
11            id=model.id,
12            name=model.name,
13            language=Language(model.language),
14            algorithm=Algorithm(model.algorithm),
15            # ... inne pola
16        )

```

Listing 2: Implementacja dla Django ORM

Zalety:

- Enkapsulacja logiki dostępu do danych
- Możliwość łatwej zamiany źródła danych (SQLite → PostgreSQL)
- Ułatwienie testowania (mock repositories)

3.3 Adapter Pattern

Cel: Umożliwienie współpracy niezgodnych interfejsów.

Implementacja - Universal RNG Adapter:

System używa wzorca Adapter do automatycznego wykrywania i adaptacji różnych implementacji generatorów:

```
1 class UniversalRNGAdapter:
2     def __init__(self, module, parameters: Optional[Dict] = None):
3         self.module = module
4         self.parameters = parameters or {}
5         self.generator_func = self._detect_generator_function()
6         self.generator_type = self._detect_generator_type()
7
8     def _detect_generator_function(self):
9         # Priorytet 1: generate()
10        if hasattr(self.module, 'generate'):
11            return self.module.generate
12
13        # Priorytet 2: *_bit_stream()
14        for name in dir(self.module):
15            if name.endswith('_bit_stream'):
16                return getattr(self.module, name)
17
18        raise RuntimeError("No generator function found")
19
20    def _detect_generator_type(self) -> DataType:
21        # Wykrywa typ danych: BITS, INTEGERS, FLOATS
22        # ...
```

Listing 3: Universal RNG Adapter

Zalety:

- Automatyczne dopasowanie do różnych sygnatur funkcji
- Obsługa różnych typów danych wyjściowych (bity, integer, floaty)
- Brak konieczności modyfikacji istniejących generatorów

3.4 Strategy Pattern

Cel: Umożliwienie wyboru algorytmu w czasie wykonania.

Implementacja - RNG Runners:

```
1 class IRNGRunner(ABC):
2     @abstractmethod
3     def can_run(self, rng: RNG) -> bool:
4         pass
5
6     @abstractmethod
7     def generate_numbers(self, rng: RNG, count: int,
8                           seed: int = None) -> List[float]:
9         pass
```

Listing 4: Interface strategii

```

1 class PythonRNGRunner(IRNGRunner):
2     def can_run(self, rng: RNG) -> bool:
3         return rng.language == Language.PYTHON
4     # ...
5
6 class ExeRNGRunner(IRNGRunner):
7     def can_run(self, rng: RNG) -> bool:
8         return rng.language == Language.EXECUTABLE
9     # ...

```

Listing 5: Konkretne strategie

Wybór strategii w runtime:

```

1 def _find_runner(self, rng: RNG) -> IRNGRunner:
2     for runner in self.runners:
3         if runner.can_run(rng):
4             return runner
5     raise RuntimeError(f"No runner for {rng.language}")

```

3.5 Use Case Pattern

Cel: Enkapsulacja logiki biznesowej w pojedynczych, specyficznych przypadkach użycia.

Implementacja:

```
1 class RunRNGTestUseCase:
2     def __init__(self,
3                 rng_repository: IRNGRepository,
4                 result_repository: ITestResultRepository,
5                 runners: List[IRNGRunner]):
6         self.rng_repository = rng_repository
7         self.result_repository = result_repository
8         self.runners = runners
9
10    def execute(self, rng_id: int, test_name: str,
11                samples_count: int, seed: int = None) -> TestResult:
12        # 1. Pobierz RNG z repozytorium
13        rng = self.rng_repository.get_by_id(rng_id)
14
15        # 2. Znajdz odpowiedni runner
16        runner = self._find_runner(rng)
17
18        # 3. Generuj liczby losowe
19        numbers = runner.generate_numbers(rng, samples_count, seed)
20
21        # 4. Wykonaj test statystyczny
22        test_result = self._perform_statistical_test(...)
23
24        # 5. Zapisz wynik
25        return self.result_repository.save(result)
```

Listing 6: Run RNG Test Use Case

Inne Use Cases w systemie:

- CompareRNGsUseCase - porównywanie generatorów
- GetTestResultsUseCase - pobieranie wyników testów
- TestCustomBitsUseCase - testowanie własnych sekwencji bitów

3.6 Dependency Injection

Cel: Odwrócenie kontroli, zwiększenie testowalności.

Implementacja w ViewSet:

```
1 class RNGViewSet(viewsets.ViewSet):
2     def __init__(self, **kwargs):
3         super().__init__(**kwargs)
4         # Tworzenie konkretnych implementacji
5         self.rng_repository = DjangoRNGRepository()
6         self.result_repository = DjangoTestResultRepository()
7         self.runners = [
8             PythonRNGRunner(),
9             ExeRNGRunner()
10        ]
11
12    @action(detail=True, methods=['post'])
13    def run_test(self, request, pk=None):
```

```
14         # Wstrzykiwanie do Use Case
15         use_case = RunRNGTestUseCase(
16             self.rng_repository,
17             self.result_repository,
18             self.runners
19         )
20         result = use_case.execute(...)
21         return Response(...)
```

Listing 7: Wstrzykiwanie zależności

3.7 Factory Pattern (Implicit)

System używa niejawnego wzorca Factory w procesie tworzenia encji z modeli Django:

```
1 def _to_entity(self, model: RNGModel) -> RNG:
2     """Factory method - creates entity from Django model"""
3     return RNG(
4         id=model.id,
5         name=model.name,
6         language=Language(model.language),
7         algorithm=Algorithm(model.algorithm),
8         description=model.description,
9         code_path=model.code_path,
10        parameters=model.parameters or {},
11        is_active=model.is_active
12    )
```

Listing 8: Konwersja Model do Entity

4 Zasady Programowania Obiektowego

4.1 SOLID Principles

System IO_RNG konsekwentnie stosuje zasady SOLID:

4.1.1 Single Responsibility Principle (SRP)

Każda klasa ma jedną, dobrze zdefiniowaną odpowiedzialność:

Przykłady:

- RNG - reprezentuje generator (encja domeny)
- DjangoRNGRepository - zarządza persystencją RNG
- PythonRNGRunner - wykonuje generatory Python
- RunRNGTestUseCase - orkiestruje proces testowania
- UniversalRNGAdapter - adaptuje różne formaty generatorów

4.1.2 Open/Closed Principle (OCP)

Klasy są otwarte na rozszerzenia, zamknięte na modyfikacje:

```
1 # Adding support for new language without modifying existing code
2 class RustRNGRunner(IRNGRunner):
3     def can_run(self, rng: RNG) -> bool:
4         return rng.language == Language.RUST
5
6     def generate_numbers(self, rng: RNG, ...) -> List[float]:
7         # Rust implementation
8         pass
9
10 # In ViewSet - just add to list
```

```

11 self.runners = [
12     PythonRNGRunner(),
13     ExeRNGRunner(),
14     RustRNGRunner() # New runner
15 ]

```

Listing 9: Przykład OCP - dodanie nowego runnera

4.1.3 Liskov Substitution Principle (LSP)

Wszystkie implementacje interfejsów są podstawialne:

```

1 # Every IRNGRunner implementation is substitutable
2 def test_with_any_runner(runner: IRNGRunner, rng: RNG):
3     if runner.can_run(rng):
4         numbers = runner.generate_numbers(rng, 1000)
5         # Works with PythonRNGRunner, ExeRNGRunner, etc.

```

4.1.4 Interface Segregation Principle (ISP)

Interfejsy są minimalistyczne i skoncentrowane:

- IRNGRepository - tylko operacje na RNG
- ITestResultRepository - tylko operacje na wynikach
- IRNGRunner - tylko generowanie liczb

4.1.5 Dependency Inversion Principle (DIP)

Moduły wysokopoziomowe zależą od abstrakcji, nie od konkretnych implementacji:

```

1 class RunRNGTestUseCase:
2     # Depends on abstractions (interfaces)
3     def __init__(self,
4         rng_repository: IRNGRepository, # Abstraction
5         result_repository: ITestResultRepository, #
6         Abstraction
7         runners: List[IRNGRunner]): # Abstraction
8         # ...

```

4.2 Enkapsulacja

Enkapsulacja danych i logiki w encjach:

```
1 @dataclass
2 class RNG:
3     # Public attributes (dataclass)
4     name: str
5     language: Language
6     algorithm: Algorithm
7     # ...
8
9     def __post_init__(self):
10         """Private validation - called automatically"""
11         if not self.name:
12             raise ValueError("RNG name cannot be empty")
13         if not self.code_path:
14             raise ValueError("Code path cannot be empty")
15
16     def validate_for_execution(self) -> bool:
17         """Public business method"""
18         return self.is_active and bool(self.code_path)
19
20     def get_parameter(self, key: str, default: Any = None) -> Any:
21         """Safe access to parameters"""
22         return self.parameters.get(key, default)
```

Listing 10: Enkapsulacja w RNG Entity

4.3 Polimorfizm

Polimorfizm w runnerach:

```
1 # Common interface
2 numbers = runner.generate_numbers(rng, count, seed)
3
4 # PythonRNGRunner - loads Python module and executes
5 # ExeRNGRunner - runs .exe file and parses stdout
6 # CppRNGRunner - compiles and runs C++ code
7 # All return List[float]
```

Listing 11: Różne implementacje tego samego interfejsu

4.4 Abstrakcja

System używa abstrakcji na wielu poziomach:

- **Encje** - abstrakcja koncepcji biznesowych (RNG, TestResult)
- **Interfejsy** - abstrakcja operacji (IRNGRunner, IRNGRepository)
- **Enumy** - abstrakcja wartości (Language, Algorithm, DataType)

5 Struktury Danych

5.1 Encje Domenowe (Dataclasses)

System używa Python dataclasses jako immutable value objects:

```
1 @dataclass
2 class RNG:
3     name: str
4     language: Language
5     algorithm: Algorithm
6     description: str
7     code_path: str
8     parameters: Optional[Dict[str, Any]] = None
9     id: Optional[int] = None
10    is_active: bool = True
```

Listing 12: Struktura encji RNG

Zalety dataclasses:

- Automatyczna generacja `__init__`, `__repr__`, `__eq__`
- Type hints dla wszystkich pól
- Niezmienność (immutability) z `frozen=True`
- Walidacja w `__post_init__`

5.2 Enumy (Typy wyliczeniowe)

System definiuje silnie typowane enumy dla wartości biznesowych:

```
1 class Language(Enum):
2     PYTHON = "python"
3     JAVA = "java"
4     CPP = "cpp"
5     RUST = "rust"
6     EXECUTABLE = "executable"
7
8 class Algorithm(Enum):
9     LINEAR_CONGRUENTIAL = "linear_congruential"
10    MERSENNE_TWISTER = "mersenne_twister"
11    XORSHIFT = "xorshift"
12    PCG = "pcg"
13    SYSTEM = "system"
14
15 class DataType(Enum):
16    BITS = "bits"
17    INTEGERS = "integers"
18    FLOATS = "floats"
```

Listing 13: Enumy domenowe

Zalety enumów:

- Bezpieczeństwo typów
- Niemożliwość użycia nieprawidłowych wartości

- Autocomplete w IDE
- Czytelność kodu

5.3 Słowniki (Dict) jako Parametry

System używa słowników JSON do przechowywania dynamicznych parametrów:

```

1 # Example LCG parameters
2 parameters = {
3     "a": 1103515245,      # multiplier
4     "c": 12345,          # increment
5     "m": 2**31,          # modulus
6     "bits_per_value": 31 # number of bits
7 }
8
9 # Safe access via method
10 def get_parameter(self, key: str, default: Any = None) -> Any:
11     return self.parameters.get(key, default) if self.parameters else
    default

```

Listing 14: Parametry generatorow

5.4 Listy typowane

System używa list z type hints dla bezpieczeństwa typów:

```

1 # List of bits
2 bits: List[int] = [0, 1, 0, 1, 1, 0, ...]
3
4 # List of floats
5 numbers: List[float] = [0.234, 0.891, 0.456, ...]
6
7 # List of entities
8 rngs: List[RNG] = repository.get_all()
9
10 # List of runners
11 runners: List[IRNGRunner] = [PythonRNGRunner(), ExeRNGRunner()]

```

5.5 Tuple dla zwracania wielu wartości

```

1 def generate_raw(self, rng: RNG, count: int,
2                 seed: int = None) -> Tuple[List[Any], DataType]:
3     """Returns (data, data_type)"""
4     # ...
5     return (data, DataType.BITS)

```

Listing 15: Tuple z type hints

5.6 Optional dla wartości nullable

```

1 def get_by_id(self, rng_id: int) -> Optional[RNG]:
2     """Can return RNG or None"""
3     try:
4         model = RNGModel.objects.get(id=rng_id)

```

```

5         return self._to_entity(model)
6     except RNGModel.DoesNotExist:
7         return None

```

5.7 Struktury danych w bazie

5.7.1 Model RNG (Tabela rngs)

```

1 class RNGModel(models.Model):
2     name = models.CharField(max_length=200)
3     language = models.CharField(max_length=50)
4     algorithm = models.CharField(max_length=100)
5     description = models.TextField()
6     code_path = models.CharField(max_length=500)
7     parameters = models.JSONField(null=True, blank=True)
8     is_active = models.BooleanField(default=True)
9     created_at = models.DateTimeField(auto_now_add=True)
10
11     class Meta:
12         db_table = 'rngs'
13         ordering = ['-created_at']

```

Listing 16: Django Model

Indeksy:

- Primary key na id
- Ordering index na created_at

5.7.2 Model TestResult (Tabela test_results)

```

1 class TestResultModel(models.Model):
2     rng = models.ForeignKey(RNGModel, on_delete=models.CASCADE,
3                             related_name='test_results')
4     test_name = models.CharField(max_length=100)
5     passed = models.BooleanField()
6     score = models.FloatField()
7     execution_time_ms = models.FloatField()
8     samples_count = models.IntegerField()
9     statistics = models.JSONField()
10    error_message = models.TextField(null=True, blank=True)
11    created_at = models.DateTimeField(auto_now_add=True)
12
13    class Meta:
14        db_table = 'test_results'
15        ordering = ['-created_at']
16        indexes = [
17            models.Index(fields=['rng', '-created_at']),
18            models.Index(fields=['test_name', '-created_at']),
19        ]

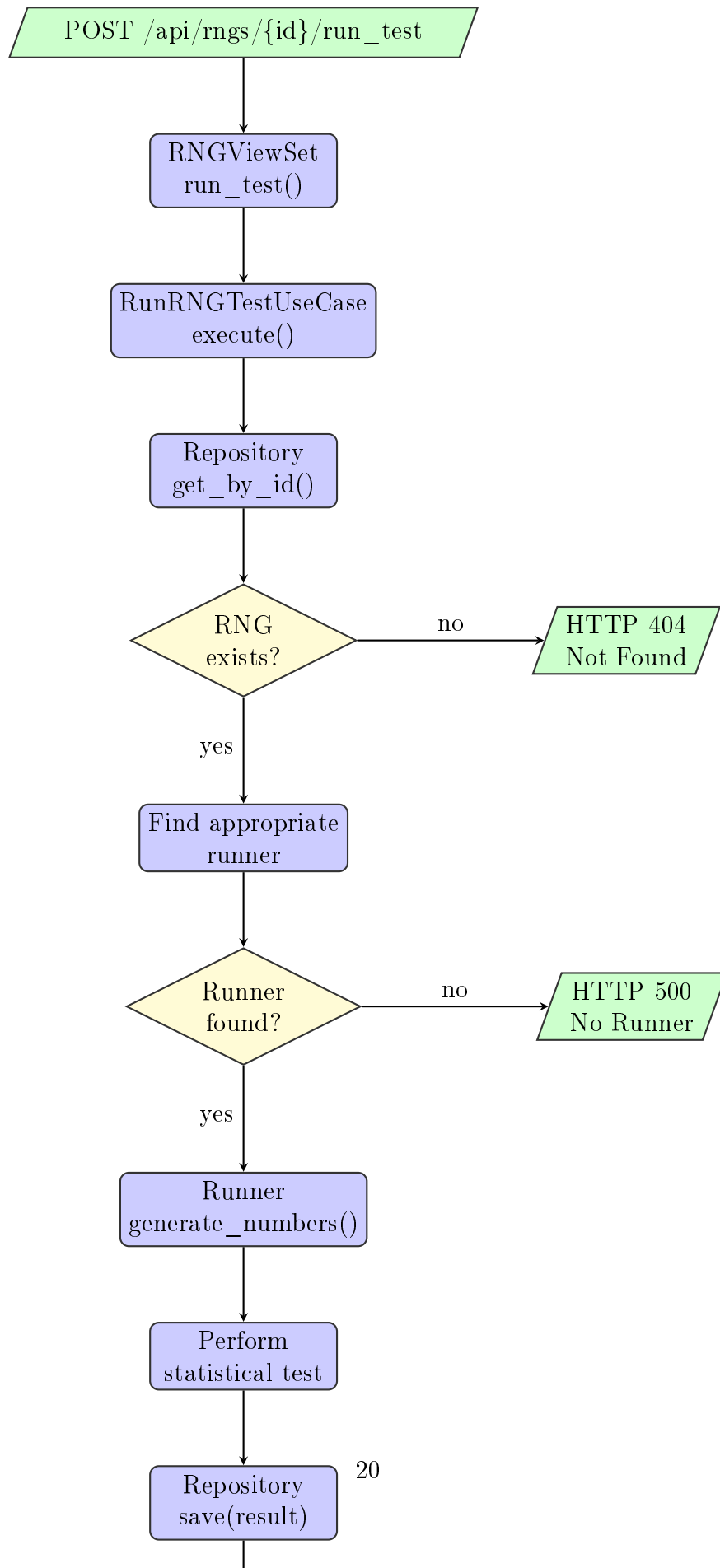
```

Indeksy optymalizacyjne:

- Composite index: (rng_id, created_at) - szybkie pobieranie wyników dla RNG
- Composite index: (test_name, created_at) - filtrowanie po typie testu

6 Diagram Przepływu Danych

6.1 Proces uruchomienia testu RNG



6.2 Proces generowania liczb - Python Runner

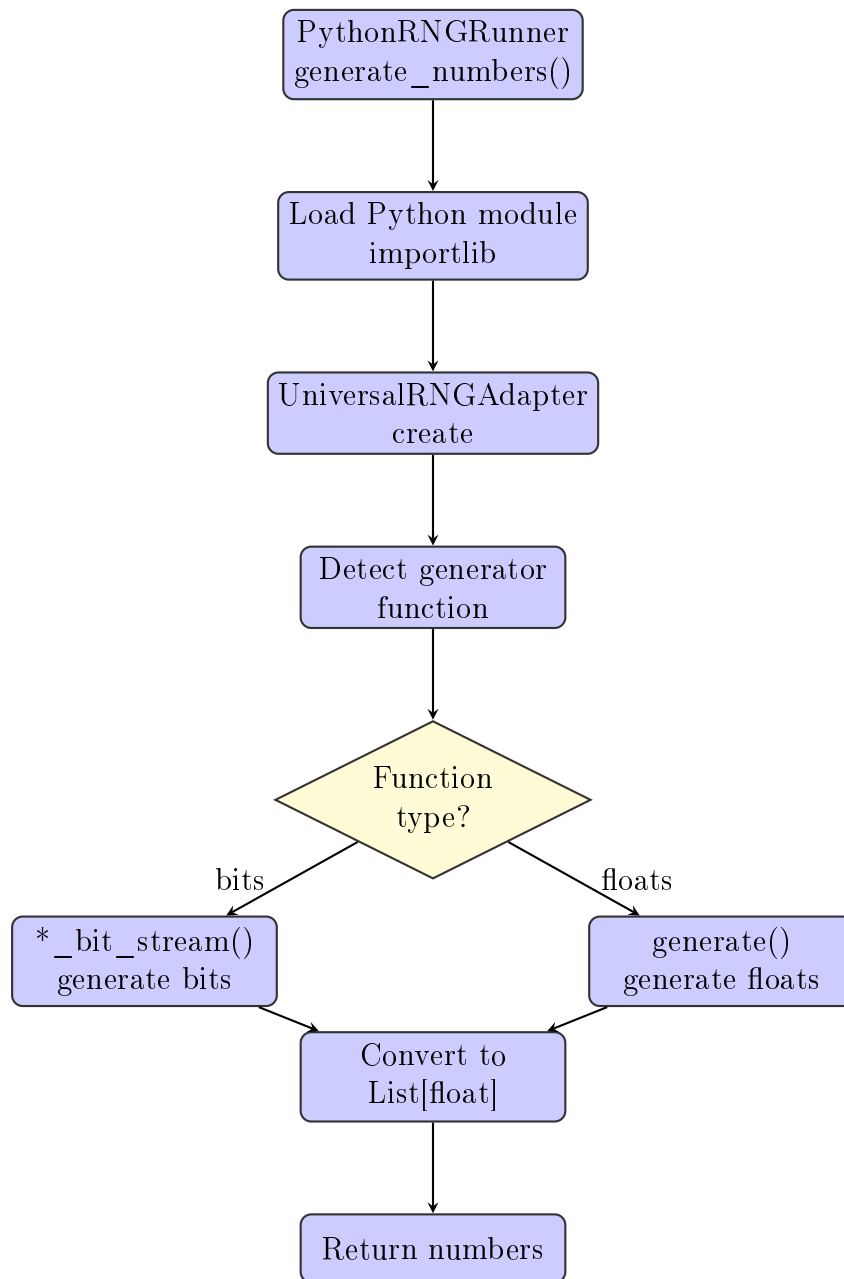


Figure 4: Proces generowania liczb w Python Runner

7 Szczegóły Implementacji

7.1 Kompresja danych - Base64

System używa kompresji bitów do Base64 dla oszczędności transferu danych:

```
1 def compress_bits_to_base64(bits: List[int]) -> str:
2     """
3     Packs bits list [0,1,0,1,...] to base64.
4     Savings: ~8x (3 bytes/bit to 0.375 bytes/bit)
5     """
```

```

6     byte_array = bytearray()
7
8     for i in range(0, len(bits), 8):
9         byte_val = 0
10        for j in range(8):
11            if i + j < len(bits):
12                # MSB first
13                byte_val |= (bits[i + j] << (7 - j))
14            byte_array.append(byte_val)
15
16    return base64.b64encode(byte_array).decode('ascii')

```

Listing 17: Kompresja bitów

Przykład:

- Bez kompresji: [1,0,1,0,1,0,1,0] → 24 bajty JSON
- Z kompresją: "qg==" → 4 bajty
- Oszczędność: 83%

7.2 Testy Statystyczne

System implementuje szereg testów statystycznych:

1. **Chi-Square Test** - test zgodności rozkładu
2. **Kolmogorov-Smirnov Test** - test równomierności
3. **Runs Test** - test niezależności sekwencji
4. **Spectral Test (FFT)** - test częstotliwościowy
5. **Poker Test** - test sekwencji 5-bitowych
6. **Monobit Test** - test proporcji bitów
7. **Serial Test** - test par bitów
8. **Autocorrelation Test** - test autokorelacji

7.3 Obsługa różnych języków programowania

7.3.1 Python Runner

- Dynamiczne ładowanie modułów: `importlib.util`
- Automatyczna detekcja funkcji generatora
- Wsparcie dla parametryzowanych generatorów (LCG, AWCG)

7.3.2 Executable Runner

- Uruchamianie prekompilowanych plików .exe
- Komunikacja przez stdout/stdin
- Parsowanie wyjścia tekstowego
- Wsparcie dla Rust, C++, C#

8 Architektura Backendowa

8.1 Przegląd struktury projektu Django

Backend systemu IO_RNG zbudowany jest w oparciu o framework Django REST Framework, stosując zasady Clean Architecture. Projekt podzielony jest na wyraźnie oddzielone warstwy, co zapewnia separację odpowiedzialności i łatwość testowania.

Struktura katalogów backendu

```
backend/
  io_rng/                                # Główna aplikacja Django
    core/                                # Warstwa domeny (Core Layer)
      entities/                          # Encje domenowe
        rng.py                           # RNG (Generator)
        test_result.py                   # Wynik testu
        enums.py                         # Language, Algorithm, DataType
      interfaces/                        # Interfejsy (ABC)
        repository.py                   # IRNGRepository, ITestResultRepository
        runner.py                       # IRNGRunner
      use_cases/                         # Logika biznesowa
        run_rng_test.py                  # Uruchomienie testu
        compare_rngs.py                  # Porównanie generatorów
        get_test_results.py              # Pobieranie wyników
        test_custom_bits.py              # Test własnych bitów

  infrastructure/                       # Warstwa infrastruktury
    models.py                           # Django ORM Models
    repositories/                       # Implementacje repozytoriów
      django_repositories.py
    runners/                            # Implementacje runnerów
      python_runner.py                   # Python RNG Runner
      exe_runner.py                      # Executable Runner
      universal_adapter.py               # Uniwersalny adapter

  api/                                  # Warstwa API
    views.py                             # Django REST Framework ViewSets
    serializers.py                       # DRF Serializers
    urls.py                              # Routing API

  settings.py                           # Konfiguracja Django
  urls.py                               # Główny routing
  wsgi.py                               # WSGI application

  db.sqlite3                            # Baza danych SQLite
  manage.py                             # Django CLI
  requirements.txt                       # Zależności Python
```


8.2 Core Layer - Warstwa Domeny

Warstwa Core zawiera całą logikę biznesową systemu i jest całkowicie niezależna od frameworka Django. Wszystkie komponenty w tej warstwie operują na czystych obiektach Pythona (dataclasses, enums, abstrakcyjne klasy).

8.2.1 Encje Domenowe

RNG Entity - reprezentuje generator liczb pseudolosowych:

```
1 @dataclass
2 class RNG:
3     """Generator liczb pseudolosowych - encja domenowa"""
4     name: str # Nazwa generatora
5     language: Language # Język implementacji
6     algorithm: Algorithm # Algorytm RNG
7     description: str # Opis generatora
8     code_path: str # Ścieżka do pliku
9     parameters: Optional[Dict] = None # Parametry (LCG: a,c,m)
10    id: Optional[int] = None # ID w bazie
11    is_active: bool = True # Czy aktywny
12
13    def __post_init__(self):
14        """Walidacja danych po inicjalizacji"""
15        if not self.name:
16            raise ValueError("RNG name cannot be empty")
17        if not self.code_path:
18            raise ValueError("Code path cannot be empty")
19
20    def validate_for_execution(self) -> bool:
21        """Sprawdza czy generator jest gotowy do wykonania"""
22        return self.is_active and bool(self.code_path)
23
24    def get_parameter(self, key: str, default: Any = None) -> Any:
25        """Bezpieczny dostęp do parametrów"""
26        return self.parameters.get(key, default) if self.parameters else default
```

Listing 18: Encja RNG

TestResult Entity - reprezentuje wynik testu statystycznego:

```
1 @dataclass
2 class TestResult:
3     """Wynik testu statystycznego - encja domenowa"""
4     rng_id: int # ID testowanego RNG
5     test_name: str # Nazwa testu
6     passed: bool # Czy test przeszedł
7     score: float # Wynik (0-1)
8     execution_time_ms: float # Czas wykonania
9     samples_count: int # Liczba próbek
10    statistics: Dict[str, Any] # Szczegółowe statystyki
11    seed: Optional[int] = None # Seed użyty do testu
12    test_parameters: Optional[Dict] = None # Parametry testu
13    error_message: Optional[str] = None # Komunikat błędu
14    id: Optional[int] = None # ID w bazie
15
16    def __post_init__(self):
17        """Walidacja wyniku testu"""
```

```

18         if not 0.0 <= self.score <= 1.0:
19             raise ValueError("Score must be between 0 and 1")
20         if self.execution_time_ms < 0:
21             raise ValueError("Execution time cannot be negative")

```

Listing 19: Encja TestResult

8.2.2 Interfejsy (Porty)

System definiuje abstrakcyjne interfejsy, które muszą być implementowane przez warstwę Infrastructure:

```

1 class IRNGRepository(ABC):
2     """Interface repozytorium generatorów RNG"""
3
4     @abstractmethod
5     def save(self, rng: RNG) -> RNG:
6         """Zapisuje generator (create lub update)"""
7         pass
8
9     @abstractmethod
10    def get_by_id(self, rng_id: int) -> Optional[RNG]:
11        """Pobiera generator po ID"""
12        pass
13
14    @abstractmethod
15    def get_all(self) -> List[RNG]:
16        """Pobiera wszystkie generatory"""
17        pass
18
19    @abstractmethod
20    def delete(self, rng_id: int) -> bool:
21        """Usuwa generator"""
22        pass
23
24 class ITestResultRepository(ABC):
25     """Interface repozytorium wyników testów"""
26
27     @abstractmethod
28     def save(self, result: TestResult) -> TestResult:
29         pass
30
31     @abstractmethod
32     def get_by_rng(self, rng_id: int) -> List[TestResult]:
33         pass
34
35     @abstractmethod
36     def get_all(self) -> List[TestResult]:
37         pass

```

Listing 20: Interfejsy repozytoriów

```

1 class IRNGRunner(ABC):
2     """Interface runnera dla różnych języków programowania"""
3
4     @abstractmethod
5     def can_run(self, rng: RNG) -> bool:
6         """Sprawdza czy runner może uruchomić dany RNG"""

```

```

7         pass
8
9     @abstractmethod
10    def generate_numbers(self, rng: RNG, count: int,
11                        seed: int = None) -> List[float]:
12        """Generuje liczby losowe z zakresu [0,1]"""
13        pass
14
15    @abstractmethod
16    def generate_raw(self, rng: RNG, count: int,
17                   seed: int = None) -> Tuple[List[Any], DataType]:
18        """Generuje surowe dane (bity, integerzy lub floaty)"""
19        pass

```

Listing 21: Interface runnera

8.2.3 Use Cases - Logika Biznesowa

Use cases orkiestrują przepływ danych między warstwami i implementują logikę biznesową:

```

1 class RunRNGTestUseCase:
2     """Przypadek użycia: Uruchomienie testu statystycznego na RNG"""
3
4     def __init__(self,
5                 rng_repository: IRNGRepository,
6                 result_repository: ITestResultRepository,
7                 runners: List[IRNGRunner]):
8         # Dependency Injection - zależności wstrzykiwane przez
konstruktor
9         self.rng_repository = rng_repository
10        self.result_repository = result_repository
11        self.runners = runners
12
13    def execute(self, rng_id: int, test_name: str,
14               samples_count: int, seed: int = None,
15               test_parameters: Dict = None) -> TestResult:
16        """
17        Wykonuje test statystyczny:
18        1. Pobiera RNG z repozytorium
19        2. Znajduje odpowiedni runner
20        3. Generuje liczby/bity
21        4. Przeprowadza test statystyczny
22        5. Zapisuje wynik
23        """
24
25        # 1. Pobranie generatora
26        rng = self.rng_repository.get_by_id(rng_id)
27        if not rng:
28            raise ValueError(f"RNG with id {rng_id} not found")
29
30        # 2. Znalezienie odpowiedniego runnera
31        runner = self._find_runner(rng)
32
33        # 3. Generowanie danych
34        start_time = time.time()
35
36        # Testy NIST wymagają bitów, inne testy floatów

```

```

37         if test_name.startswith('nist_') or test_name.startswith('
diehard_'):
38             bits, data_type = runner.generate_raw(rng, samples_count,
seed)
39             # Konwersja do bitów jeśli potrzeba
40             if data_type != DataType.BITS:
41                 bits = self._convert_to_bits(bits, data_type)
42         else:
43             # Podstawowe testy (frequency, uniformity) używają floatów
44             numbers = runner.generate_numbers(rng, samples_count, seed)
45
46         generation_time = (time.time() - start_time) * 1000
47
48         # 4. Przeprowadzenie testu statystycznego
49         test_result = self._perform_test(
50             test_name,
51             bits if 'bits' in locals() else numbers,
52             test_parameters
53         )
54
55         # 5. Utworzenie i zapisanie wyniku
56         result = TestResult(
57             rng_id=rng_id,
58             test_name=test_name,
59             passed=test_result['passed'],
60             score=test_result['score'],
61             execution_time_ms=generation_time + test_result['
test_time_ms'],
62             samples_count=samples_count,
63             statistics=test_result['statistics'],
64             seed=seed,
65             test_parameters=test_parameters
66         )
67
68         return self.result_repository.save(result)

```

Listing 22: RunRNGTestUseCase - główny przypadek użycia

8.3 Infrastructure Layer - Warstwa Infrastruktury

Warstwa Infrastructure zawiera konkretne implementacje interfejsów zdefiniowanych w Core Layer. Używa Django ORM do persystencji danych.

8.3.1 Django Models

```
1 class RNGModel(models.Model):
2     """Model Django dla generatorów RNG"""
3
4     name = models.CharField(max_length=200, unique=True)
5     language = models.CharField(max_length=50) # python, cpp, rust, etc
6
7     algorithm = models.CharField(max_length=100)
8     description = models.TextField(blank=True)
9     code_path = models.CharField(max_length=500) # Względna ścieżka
10    parameters = models.JSONField(null=True, blank=True) # Parametry
11    is_active = models.BooleanField(default=True)
12    created_at = models.DateTimeField(auto_now_add=True)
13    updated_at = models.DateTimeField(auto_now=True)
14
15    class Meta:
16        db_table = 'rngs'
17        ordering = ['-created_at']
18        indexes = [
19            models.Index(fields=['language']),
20            models.Index(fields=['algorithm']),
21        ]
22
23 class TestResultModel(models.Model):
24     """Model Django dla wyników testów"""
25
26     rng = models.ForeignKey(RNGModel, on_delete=models.CASCADE,
27                             related_name='test_results')
28     test_name = models.CharField(max_length=100)
29     passed = models.BooleanField()
30     score = models.FloatField() # 0.0 - 1.0
31     execution_time_ms = models.FloatField()
32     samples_count = models.IntegerField()
33     statistics = models.JSONField() # Szczegółowe statystyki
34     seed = models.IntegerField(null=True, blank=True)
35     test_parameters = models.JSONField(null=True, blank=True)
36     error_message = models.TextField(null=True, blank=True)
37     created_at = models.DateTimeField(auto_now_add=True)
38
39    class Meta:
40        db_table = 'test_results'
41        ordering = ['-created_at']
42        indexes = [
43            models.Index(fields=['rng', '-created_at']),
44            models.Index(fields=['test_name', '-created_at']),
45            models.Index(fields=['passed']),
46        ]
```

Listing 23: Modele Django ORM

8.3.2 Repository Implementations

Repozytoria implementują wzorzec Repository, przekształcając encje domenowe w modele Django i vice versa:

```
1 class DjangoRNGRepository(IRNGRepository):
2     """Konkretna implementacja repozytorium używająca Django ORM"""
3
4     def save(self, rng: RNG) -> RNG:
5         """Zapisuje lub aktualizuje RNG"""
6         if rng.id:
7             # Update istniejącego
8             model = RNGModel.objects.get(id=rng.id)
9             self._update_model_from_entity(model, rng)
10        else:
11            # Utworzenie nowego
12            model = self._create_model_from_entity(rng)
13
14        model.save()
15        return self._to_entity(model)
16
17    def get_by_id(self, rng_id: int) -> Optional[RNG]:
18        """Pobiera RNG po ID, zwraca None jeśli nie istnieje"""
19        try:
20            model = RNGModel.objects.get(id=rng_id)
21            return self._to_entity(model)
22        except RNGModel.DoesNotExist:
23            return None
24
25    def _to_entity(self, model: RNGModel) -> RNG:
26        """Konwersja Django Model -> Domain Entity (Factory Method)"""
27        return RNG(
28            id=model.id,
29            name=model.name,
30            language=Language(model.language),
31            algorithm=Algorithm(model.algorithm),
32            description=model.description,
33            code_path=model.code_path,
34            parameters=model.parameters or {},
35            is_active=model.is_active
36        )
37
38    def _create_model_from_entity(self, rng: RNG) -> RNGModel:
39        """Konwersja Domain Entity -> Django Model"""
40        return RNGModel(
41            name=rng.name,
42            language=rng.language.value,
43            algorithm=rng.algorithm.value,
44            description=rng.description,
45            code_path=rng.code_path,
46            parameters=rng.parameters,
47            is_active=rng.is_active
48        )
```

Listing 24: Implementacja DjangoRNGRepository

8.3.3 System Runnerów

System runnerów umożliwia wykonywanie generatorów napisanych w różnych językach programowania. Każdy runner implementuje interfejs `IRNGRunner`.

Obsługiwane języki

- **Python** - dynamiczne ładowanie modułów (`importlib`)
- **C++** - kompilacja i uruchomienie plików `.exe`
- **Rust** - uruchomienie prekompilowanych binarek
- **C#** - uruchomienie plików `.exe/.dll`
- **Java** - kompilacja i uruchomienie `.class` (TODO)

Python RNG Runner:

```
1 class PythonRNGRunner(IRNGRunner):
2     """Runner dla generatorów napisanych w Pythonie"""
3
4     BASE_PATH = Path(__file__).parent.parent.parent / "algorytmy"
5
6     def can_run(self, rng: RNG) -> bool:
7         """Sprawdza czy runner obsługuje dany RNG"""
8         return rng.language == Language.PYTHON
9
10    def generate_numbers(self, rng: RNG, count: int,
11                        seed: int = None) -> List[float]:
12        """Generuje liczby float [0,1]"""
13        # 1. Załaduj moduł Python
14        module = self._load_module(rng.code_path)
15
16        # 2. Utwórz Universal Adapter
17        adapter = UniversalRNGAdapter(module, rng.parameters)
18
19        # 3. Wygeneruj surowe dane
20        raw_data, data_type = adapter.generate_raw(count, seed)
21
22        # 4. Konwertuj do floatów [0,1]
23        return self._convert_to_floats(raw_data, data_type)
24
25    def _load_module(self, code_path: str):
26        """Dynamicznie ładuje moduł Python używając importlib"""
27        full_path = self.BASE_PATH / code_path
28
29        spec = importlib.util.spec_from_file_location(
30            "rng_module", full_path
31        )
32        module = importlib.util.module_from_spec(spec)
33        spec.loader.exec_module(module)
34
35        return module
```

Listing 25: PythonRNGRunner - wykonywanie generatorów Python

Executable RNG Runner:

```

1 class ExeRNGRunner(IRNGRunner):
2     """Runner dla prekompilowanych plików wykonywalnych (C++, Rust, C#)"""
3
4     def can_run(self, rng: RNG) -> bool:
5         return rng.language == Language.EXECUTABLE
6
7     def generate_raw(self, rng: RNG, count: int,
8                     seed: int = None) -> Tuple[List[Any], DataType]:
9         """Uruchamia plik .exe i parsuje stdout"""
10
11        # 1. Znajdź plik wykonywalny
12        exe_path = self._resolve_executable_path(rng.code_path)
13
14        # 2. Przygotuj argumenty
15        args = [str(exe_path), str(count)]
16        if seed is not None:
17            args.append(str(seed))
18
19        # 3. Uruchom proces
20        result = subprocess.run(
21            args,
22            capture_output=True,
23            text=True,
24            timeout=30 # 30s timeout
25        )
26
27        if result.returncode != 0:
28            raise RuntimeError(f"Executable failed: {result.stderr}")
29
30        # 4. Parsuj wyjście (format: liczba na linię lub JSON)
31        output = result.stdout.strip()
32
33        # Wykryj format wyjścia
34        if output.startswith('[') or output.startswith('{'):
35            # Format JSON
36            data = json.loads(output)
37            return data['bits'], DataType.BITS
38        else:
39            # Format tekstowy - liczby w liniach
40            numbers = [float(line) for line in output.split('\n') if
41line]
42
43            return numbers, DataType.FLOATS

```

Listing 26: ExeRNGRunner - uruchamianie plików wykonywalnych

Universal RNG Adapter:

Adapter automatycznie wykrywa typ funkcji generatora i dostosowuje się do jej sygnatury:

```

1 class UniversalRNGAdapter:
2     """
3     Uniwersalny adapter dla różnych typów generatorów.
4     Automatycznie wykrywa:
5     - Typ funkcji (generate, *_bit_stream, etc.)
6     - Typ danych wyjściowych (bits, integers, floats)
7     - Wymagane parametry
8     """
9
10    def __init__(self, module, parameters: Optional[Dict] = None):

```



```

11     self.module = module
12     self.parameters = parameters or {}
13     self.generator_func = self._detect_generator_function()
14     self.data_type = self._detect_data_type()
15
16     def _detect_generator_function(self):
17         """Wykrywa funkcję generatora w module"""
18         # Priorytet 1: funkcja generate()
19         if hasattr(self.module, 'generate'):
20             return self.module.generate
21
22         # Priorytet 2: funkcje *_bit_stream()
23         for name in dir(self.module):
24             if name.endswith('_bit_stream'):
25                 return getattr(self.module, name)
26
27         # Priorytet 3: pierwsza funkcja publiczna
28         for name in dir(self.module):
29             if not name.startswith('_'):
30                 attr = getattr(self.module, name)
31                 if callable(attr):
32                     return attr
33
34         raise RuntimeError("No generator function found in module")
35
36     def _detect_data_type(self) -> DataType:
37         """Wykrywa typ danych zwracanych przez generator"""
38         # Sprawdź nazwę funkcji
39         func_name = self.generator_func.__name__
40
41         if 'bit' in func_name.lower():
42             return DataType.BITS
43         elif 'int' in func_name.lower():
44             return DataType.INTEGERS
45         else:
46             return DataType.FLOATS
47
48     def generate_raw(self, count: int, seed: int = None) -> Tuple[List,
49     DataType]:
50         """Generuje surowe dane odpowiadając na sygnaturę funkcji"""
51
52         # Sprawdź sygnaturę funkcji
53         sig = inspect.signature(self.generator_func)
54
55         # Przygotuj argumenty
56         kwargs = {}
57
58         if 'count' in sig.parameters or 'n' in sig.parameters:
59             kwargs['count'] = count
60         if 'seed' in sig.parameters:
61             kwargs['seed'] = seed
62
63         # Dodaj parametry generatora (dla LCG: a, c, m)
64         for param_name in sig.parameters:
65             if param_name in self.parameters:
66                 kwargs[param_name] = self.parameters[param_name]
67
68         # Wywołaj funkcję

```

```
68     data = self.generator_func(**kwargs)
69
70     return list(data), self.data_type
```

Listing 27: UniversalRNGAdapter - automatyczna detekcja

8.4 API Layer - Warstwa Prezentacji

Warstwa API używa Django REST Framework do udostępniania funkcjonalności systemu przez RESTful API.

8.4.1 ViewSets

```
1 class RNGViewSet(viewsets.ViewSet):
2     """ViewSet obsługujący operacje CRUD na generatorach RNG"""
3
4     def __init__(self, **kwargs):
5         super().__init__(**kwargs)
6         # Dependency Injection - tworzenie zależności
7         self.rng_repository = DjangoRNGRepository()
8         self.result_repository = DjangoTestResultRepository()
9         self.runners = [
10             PythonRNGRunner(),
11             ExeRNGRunner()
12         ]
13
14     def list(self, request):
15         """GET /api/rngs - lista wszystkich generatorów"""
16         rngs = self.rng_repository.get_all()
17         serializer = RNGSerializer(rngs, many=True)
18         return Response(serializer.data)
19
20     def retrieve(self, request, pk=None):
21         """GET /api/rngs/{id} - szczegóły generatora"""
22         rng = self.rng_repository.get_by_id(int(pk))
23         if not rng:
24             return Response(
25                 {"error": "RNG not found"},
26                 status=status.HTTP_404_NOT_FOUND
27             )
28         serializer = RNGSerializer(rng)
29         return Response(serializer.data)
30
31     def create(self, request):
32         """POST /api/rngs - utworzenie nowego generatora"""
33         serializer = RNGSerializer(data=request.data)
34         serializer.is_valid(raise_exception=True)
35
36         rng = serializer.save_to_entity()
37         saved_rng = self.rng_repository.save(rng)
38
39         return Response(
40             RNGSerializer(saved_rng).data,
41             status=status.HTTP_201_CREATED
42         )
43
44     @action(detail=True, methods=['post'])
45     def run_test(self, request, pk=None):
46         """
47         POST /api/rngs/{id}/run_test
48         Uruchamia test statystyczny na generatorze
49         """
50         # Walidacja danych wejściowych
```

```

51     test_name = request.data.get('test_name')
52     samples_count = request.data.get('samples_count', 10000)
53     seed = request.data.get('seed')
54     test_parameters = request.data.get('test_parameters')
55
56     # Wywołanie Use Case
57     use_case = RunRNGTestUseCase(
58         self.rng_repository,
59         self.result_repository,
60         self.runners
61     )
62
63     try:
64         result = use_case.execute(
65             rng_id=int(pk),
66             test_name=test_name,
67             samples_count=samples_count,
68             seed=seed,
69             test_parameters=test_parameters
70         )
71
72         serializer = TestResultSerializer(result)
73         return Response(serializer.data)
74
75     except Exception as e:
76         return Response(
77             {"error": str(e)},
78             status=status.HTTP_500_INTERNAL_SERVER_ERROR
79         )
80
81 @action(detail=True, methods=['post'])
82 def generate(self, request, pk=None):
83     """
84     POST /api/rngs/{id}/generate
85     Generuje surowe bity bez przeprowadzania testów
86     """
87     count = request.data.get('count', 10000)
88     seed = request.data.get('seed')
89
90     rng = self.rng_repository.get_by_id(int(pk))
91     if not rng:
92         return Response(
93             {"error": "RNG not found"},
94             status=status.HTTP_404_NOT_FOUND
95         )
96
97     # Znajdź runner i wygeneruj
98     runner = self._find_runner(rng)
99
100     start_time = time.time()
101     bits, data_type = runner.generate_raw(rng, count, seed)
102     execution_time = (time.time() - start_time) * 1000
103
104     return Response({
105         "bits": bits,
106         "count": len(bits),
107         "execution_time_ms": execution_time,
108         "rng_id": rng.id,

```

```

109         "rng_name": rng.name,
110         "seed": seed
111     })

```

Listing 28: RNGViewSet - główny endpoint dla RNG

8.4.2 Serializers

Serializatory konwertują encje domenowe do/z formatu JSON:

```

1 class RNGSerializer(serializers.Serializer):
2     """Serializer dla encji RNG"""
3
4     id = serializers.IntegerField(read_only=True)
5     name = serializers.CharField(max_length=200)
6     language = serializers.ChoiceField(
7         choices=[(lang.value, lang.name) for lang in Language]
8     )
9     algorithm = serializers.ChoiceField(
10        choices=[(alg.value, alg.name) for alg in Algorithm]
11    )
12    description = serializers.CharField(allow_blank=True)
13    code_path = serializers.CharField(max_length=500)
14    parameters = serializers.JSONField(required=False, allow_null=True)
15    is_active = serializers.BooleanField(default=True)
16    created_at = serializers.DateTimeField(read_only=True)
17
18    def save_to_entity(self) -> RNG:
19        """Konwersja validated_data do encji domenowej"""
20        return RNG(
21            name=self.validated_data['name'],
22            language=Language(self.validated_data['language']),
23            algorithm=Algorithm(self.validated_data['algorithm']),
24            description=self.validated_data.get('description', ''),
25            code_path=self.validated_data['code_path'],
26            parameters=self.validated_data.get('parameters'),
27            is_active=self.validated_data.get('is_active', True)
28        )
29
30 class TestResultSerializer(serializers.Serializer):
31     """Serializer dla wyników testów"""
32
33     id = serializers.IntegerField(read_only=True)
34     rng_id = serializers.IntegerField()
35     test_name = serializers.CharField()
36     passed = serializers.BooleanField()
37     score = serializers.FloatField(min_value=0.0, max_value=1.0)
38     execution_time_ms = serializers.FloatField()
39     samples_count = serializers.IntegerField()
40     statistics = serializers.JSONField()
41     seed = serializers.IntegerField(allow_null=True, required=False)
42     test_parameters = serializers.JSONField(allow_null=True, required=False)
43     created_at = serializers.DateTimeField(read_only=True)

```

Listing 29: DRF Serializers

8.5 Testy Statystyczne

System implementuje kompletne baterie testów statystycznych dla weryfikacji jakości generatorów liczb pseudolosowych.

8.5.1 NIST Statistical Test Suite

Pełna implementacja 15 testów NIST SP 800-22:

Table 1: Testy NIST - kompletna bateria

Test	Opis	Min. bitów
Monobit	Proporcja 0s i 1s	100
Block Frequency	Balans bitów w blokach	100
Runs	Liczba ciągów jedynek/zer	100
Longest Run	Najdłuższy ciąg jedynek	128
Binary Matrix Rank	Ranga macierzy binarnej	38,912
DFT (Spectral)	Analiza częstotliwości (FFT)	1,000
Non-Overlapping Template	Wzorce bez nakładania	1,000
Overlapping Template	Wzorce z nakładaniem	1,000
Universal (Maurer)	Test kompresji	387,840
Linear Complexity	Złożoność liniowa (Berlekamp-Massey)	1,000,000
Serial	Częstość m-bitowych wzorców	1,000
Approximate Entropy	Entropia przybliżona	100
Cumulative Sums	Sumy kumulacyjne	100
Random Excursions	Spacer losowy - cykle	1,000,000
Random Excursions Variant	Spacer losowy - warianty	1,000,000

8.5.2 Diehard Battery of Tests

Implementacja 15 klasycznych testów Diehard:

Table 2: Testy Diehard - kompletna bateria

Test	Opis
Birthday Spacings	Kolizje dat urodzin w próbkach
Overlapping Permutations	Analiza nakładających się permutacji
Binary Rank (31x31)	Ranga macierzy binarnej 31x31
Binary Rank (32x32)	Ranga macierzy binarnej 32x32
Binary Rank (6x8)	Ranga macierzy binarnej 6x8
Bitstream	Nakładające się 20-bitowe słowa
OPSO	Overlapping-Pairs-Sparse-Occupancy
OQSO	Overlapping-Quadruples-Sparse-Occupancy
DNA	Zliczanie 10-literowych "słów DNA"
Count the 1s (stream)	Liczenie jedynek w strumieniu
Count the 1s (byte)	Liczenie jedynek w bajtach
Parking Lot	Test parkowania - kolizje współrzędnych
Minimum Distance	Minimalna odległość między punktami
3D Spheres	Minimalna odległość punktów na sferze 3D
Squeeze	Test kompresji iteracyjnej
Overlapping Sums	Sumy nakładających się sekwencji
Runs	Analiza długości ciągów
Craps	Symulacja gry w kości

8.5.3 Implementacja testów

Wszystkie testy zwracają zunifikowany wynik:

```

1 {
2     "passed": bool,           # Czy test przeszedł (p-value > 0.01)
3     "score": float,          # Wynik 0-1 (wyższy = lepszy)
4     "statistics": {
5         "p_value": float,    # P-value testu
6         # ... inne statystyki specyficzne dla testu
7     },
8     "test_time_ms": float    # Czas wykonania testu
9 }
```

Listing 30: Struktura wyniku testu

8.6 Konfiguracja Django

8.6.1 Settings.py

Kluczowe ustawienia projektu:

```
1 # Backend Settings
2 INSTALLED_APPS = [
3     'django.contrib.admin',
4     'django.contrib.auth',
5     'django.contrib.contenttypes',
6     'django.contrib.sessions',
7     'django.contrib.messages',
8     'django.contrib.staticfiles',
9
10    # Third-party apps
11    'rest_framework',
12    'corsheaders',
13
14    # Local apps
15    'io_rng',
16 ]
17
18 MIDDLEWARE = [
19     'corsheaders.middleware.CorsMiddleware', # CORS - musi być na począ
20     'django.middleware.security.SecurityMiddleware',
21     'django.contrib.sessions.middleware.SessionMiddleware',
22     'django.middleware.common.CommonMiddleware',
23     'django.middleware.csrf.CsrfViewMiddleware',
24     'django.contrib.auth.middleware.AuthenticationMiddleware',
25     'django.contrib.messages.middleware.MessageMiddleware',
26     'django.middleware.clickjacking.XFrameOptionsMiddleware',
27 ]
28
29 # CORS Configuration - dla frontendu React
30 CORS_ALLOWED_ORIGINS = [
31     "http://localhost:5173", # Vite dev server
32     "http://127.0.0.1:5173",
33 ]
34
35 CORS_ALLOW_METHODS = [
36     'DELETE',
37     'GET',
38     'OPTIONS',
39     'PATCH',
40     'POST',
41     'PUT',
42 ]
43
44 # REST Framework Configuration
45 REST_FRAMEWORK = {
46     'DEFAULT_RENDERER_CLASSES': [
47         'rest_framework.renderers.JSONRenderer',
48     ],
49     'DEFAULT_PARSER_CLASSES': [
50         'rest_framework.parsers.JSONParser',
51     ],
52 }
```



```

53
54 # Database - SQLite
55 DATABASES = {
56     'default': {
57         'ENGINE': 'django.db.backends.sqlite3',
58         'NAME': BASE_DIR / 'db.sqlite3',
59     }
60 }

```

Listing 31: Konfiguracja Django

8.6.2 URL Routing

```

1 # backend/io_rng/urls.py
2 from django.urls import path, include
3 from rest_framework.routers import DefaultRouter
4 from .api.views import RNGViewSet, TestResultViewSet
5
6 router = DefaultRouter()
7 router.register(r'rngs', RNGViewSet, basename='rng')
8 router.register(r'test-results', TestResultViewSet, basename='testresult')
9
10 urlpatterns = [
11     path('api/', include(router.urls)),
12 ]

```

Listing 32: Konfiguracja URL

8.7 System Migracji Bazy Danych

Django używa systemu migracji do zarządzania schematem bazy danych:

```

1 # Utworzenie nowych migracji po zmianie modeli
2 python manage.py makemigrations
3
4 # Zastosowanie migracji do bazy danych
5 python manage.py migrate
6
7 # Wyświetlenie stanu migracji
8 python manage.py showmigrations
9
10 # Cofnięcie migracji
11 python manage.py migrate io_rng 0004_previous_migration

```

Listing 33: Komendy migracji

Przykład migracji:

```

1 # io_rng/infrastructure/migrations/0005_testresultmodel_test_parameters.py
2 from django.db import migrations, models
3
4 class Migration(migrations.Migration):
5     dependencies = [
6         ('io_rng', '0004_previous_migration'),
7     ]
8

```

```

9      operations = [
10          migrations.AddField(
11              model_name='testresultmodel',
12              name='test_parameters',
13              field=models.JSONField(blank=True, null=True),
14          ),
15      ]

```

Listing 34: Przykładowa migracja Django

8.8 Zalety architektury backendowej

Kluczowe zalety

1. **Clean Architecture** - pełna separacja warstw, niezależność od frameworka
2. **Testowalność** - łatwe mockowanie dzięki Dependency Injection
3. **Rozszerzalność** - łatwe dodawanie nowych runnerów, testów, algorytmów
4. **Type Safety** - type hints w całym projekcie
5. **Wielojęzyczność** - obsługa RNG w Python, C++, Rust, C#
6. **Uniwersalność** - Universal Adapter automatycznie dopasowuje się do różnych sygnatur
7. **Wydajność** - optymalizacja poprzez kompresję Base64, numpy
8. **Kompletność** - pełne baterie NIST (15 testów) i Diehard (15 testów)

9 Architektura Frontendowa

9.1 Struktura komponentów React

Frontend używa architektury komponentowej z podziałem funkcjonalnym:

Struktura komponentów

- **Layout:**
 - `NavigationSidebar` - nawigacja boczna
 - `Footer` - stopka
 - `Header` - nagłówek
- **Pages (Routes):**
 - `Dashboard` - pulpit główny
 - `Generator` - generator liczb
 - `Tests` - testy statystyczne
 - `Results` - wyniki testów
 - `Wiki` - dokumentacja algorytmów
 - `Settings` - ustawienia
- **UI Components (shadcn/ui):**
 - `Button`, `Card`, `Dialog`, `Select`
 - `Table`, `Tabs`, `Progress`
 - `Chart` (Recharts)
- **Context Providers:**
 - `ThemeProvider` - motyw (dark/light)
 - `LanguageProvider` - internacjonalizacja
 - `TestResultsProvider` - stan wyników testów

9.2 Wzorce frontendowe

9.2.1 Context API dla globalnego stanu

```
1 export const TestResultsProvider = ({ children }) => {
2   const [results, setResults] = useState<TestResult[]>([]);
3   const [loading, setLoading] = useState(false);
4
5   const fetchResults = async () => {
6     setLoading(true);
7     const data = await api.getTestResults();
8     setResults(data);
9     setLoading(false);
10  };
```

```

11
12  return (
13    <TestResultsContext.Provider value={{
14      results, loading, fetchResults
15    }}>
16      {children}
17    </TestResultsContext.Provider>
18  );
19 };

```

Listing 35: Test Results Context

9.2.2 Custom Hooks

System używa custom hooks dla logiki wielokrotnego użytku:

- `useRNGList()` - pobieranie listy generatorów
- `useTestExecution()` - uruchamianie testów
- `useLanguage()` - zarządzanie językiem interfejsu
- `useTheme()` - zarządzanie motywem

10 Komunikacja Backend-Frontend

10.1 REST API Endpoints

Metoda	Endpoint	Opis
GET	<code>/api/rngs</code>	Lista wszystkich RNG
POST	<code>/api/rngs</code>	Utworzenie nowego RNG
GET	<code>/api/rngs/{id}</code>	Szczegóły RNG
PUT	<code>/api/rngs/{id}</code>	Aktualizacja RNG
DELETE	<code>/api/rngs/{id}</code>	Usunięcie RNG
POST	<code>/api/rngs/{id}/run_test</code>	Uruchomienie testu
POST	<code>/api/rngs/{id}/generate</code>	Generowanie liczb bez testu
GET	<code>/api/rngs/{id}/test_results</code>	Wyniki testów dla RNG
GET	<code>/api/test-results</code>	Lista wszystkich wyników
GET	<code>/api/test-results/{id}</code>	Szczegóły wyniku
DELETE	<code>/api/test-results/{id}</code>	Usunięcie wyniku

Table 3: REST API Endpoints

10.2 Przykładowe zapytanie i odpowiedź

Request:

```

1 POST /api/rngs/1/run_test
2 Content-Type: application/json
3
4 {
5   "test_name": "chi_square",

```

```

6  "samples_count": 100000,
7  "seed": 42,
8  "parameters": {
9    "a": 1103515245,
10   "c": 12345,
11   "m": 2147483648
12 }
13 }

```

Response:

```

1  {
2    "id": 123,
3    "rng_id": 1,
4    "test_name": "chi_square",
5    "passed": true,
6    "score": 0.87,
7    "execution_time_ms": 234.56,
8    "samples_count": 100000,
9    "statistics": {
10     "chi_square_statistic": 12.34,
11     "p_value": 0.87,
12     "degrees_of_freedom": 9
13   },
14   "created_at": "2026-01-14T10:30:00Z"
15 }

```

Listing 36: JSON Response

11 Podsumowanie

11.1 Zastosowane wzorce architektoniczne

1. **Clean Architecture** - podział na warstwy: Core, Infrastructure, API
2. **Repository Pattern** - abstrakcja dostępu do danych
3. **Adapter Pattern** - Universal RNG Adapter
4. **Strategy Pattern** - wymienne runnery dla języków
5. **Use Case Pattern** - enkapsulacja logiki biznesowej
6. **Dependency Injection** - wstrzykiwanie zależności
7. **MVC/MVP** - separacja widoku i logiki (Frontend)

11.2 Zasady OOP

1. **SOLID:**
 - **Single Responsibility** - każda klasa ma jedną odpowiedzialność
 - **Open/Closed** - otwarte na rozszerzenia, zamknięte na modyfikacje
 - **Liskov Substitution** - implementacje podstawialne
 - **Interface Segregation** - minimalistyczne interfejsy

- **Dependency Inversion** - zależność od abstrakcji
2. **Enkapsulacja** - dane i logika ukryte w klasach
 3. **Polimorfizm** - różne implementacje tego samego interfejsu
 4. **Abstrakcja** - ukrycie szczegółów implementacji
 5. **Dziedziczenie** - przez interfejsy (ABC)

11.3 Struktury danych

1. **Dataclasses** - encje domenowe (RNG, TestResult)
2. **Enumy** - typy wyliczeniowe (Language, Algorithm, DataType)
3. **Dict** - parametry dynamiczne, JSON
4. **List[T]** - kolekcje typowane
5. **Tuple** - zwracanie wielu wartości
6. **Optional[T]** - wartości nullable
7. **Django Models** - ORM dla persystencji
8. **JSONField** - elastyczne przechowywanie danych

11.4 Zalety architektury

- **Testowalność** - łatwe mockowanie interfejsów
- **Rozszerzalność** - łatwe dodawanie nowych runnerów, testów, algorytmów
- **Niezależność od frameworka** - logika biznesowa w Core
- **Separacja warstw** - clear separation of concerns
- **Type Safety** - type hints w Python, TypeScript w frontend
- **Maintainability** - czytelna struktura, SOLID